Raspberry Pi → Collector: delivery protocol

This is the contract between the Raspberry Pi software and the data collector (`run_data_collector`). Read it before writing or changing the Pi code. The server side is described in `README.md`; the implementation is `src/collector_server.rs`, and `tests/collector_test.rs` shows every case below as an executable example.

Goals

1. **No lost events.** If the network drops, the Pi keeps the event and sends it again later.
2. **No duplicate events.** Sending the same event several times, for example because the reply was lost, stores it once.
3. **The Pi knows for certain** that the server stored exactly what the Pi sent.

How it works, in one paragraph

The Pi gives every event a unique **`event_id`** (a random, **version 4** UUID) when the event is created, and sends it inside the JSON. The server stores the event under that id and computes the **SHA-256 of the exact bytes of the request body**, which it stores and returns. The Pi computes the same SHA-256 over the bytes it sent. If the two hashes match and the status is `200`, the event is safely stored and the Pi may delete its local copy. Anything else means: keep it and try again (or see the table for the few cases where retrying will not help).

Request

```
POST https://<server>/api/alpr/event
Content-Type: application/json
X-Collector-Token: <shared secret>
```

The body is a JSON object with exactly these fields:

Field	Type	Notes
`event_id`	string	**A random (version 4) UUID, generated by the Pi once per event and reused for every retry of that event.** See "The `event_id`" below. Any standard textual form is accepted (`xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`). The server stores it lowercase with hyphens. Anything that is not a UUID is rejected with `400`. It is used in file names, which is why it is validated strictly.
`camid`	string	Must match a camera registered on the server (`camera.id`). Unknown cameras are rejected with `422`.
`date`	string	When the vehicle was seen. **RFC 3339 with an offset**, e.g. `2026-10-09T16:44:25.443+08:00` or `2026-10-09T08:44:25.443Z`. The server converts it to `+08:00` for storage.
`plate`	string	The recognised plate text.
`image`	string	The scene image, **base64** (standard alphabet, with padding), a JPEG.
`plate_image`	string	The cropped plate image, base64 JPEG.

Maximum body size is **20 MB** (larger gets `413`). Base64 makes images about a third bigger, so this allows roughly 14 MB of raw JPEG data per event in total.

The `X-Collector-Token` header is checked by nginx, not by the collector. A missing or wrong token gets `401` from nginx. Always use `https://`, because the token is only secret on an encrypted connection.

The `event_id`

Use a version 4 (random) UUID, generated from the operating system's random source. In Rust that is `Uuid::new_v4()` from the `uuid` crate with the `v4` feature, which is what the example below uses.

Why this matters: the protocol treats two events with the same `event_id` as the same event. Collisions between genuinely random v4 UUIDs are not a practical concern: a v4 UUID has 122 random bits. For example, 500 Pis each recording 5,000 events a day for 10 years (about 9 billion events) have a chance of about 1 in 10^17 of producing even one repeated id. (The chance grows with the square of the number of events, so 100 times as many events is about 10,000 times as likely, and still only roughly 1 in 10^13.) **Those figures hold only if the ids really are random.** Collisions in practice come from weak randomness or from ids that are not random at all, so:

- **Do not** derive the id from the plate, the camera, the time, or a counter (a counter that resets on reboot repeats immediately).

- **Do not** use UUID versions 1, 3, 5, 6, 7 or 8. v1 and v6 embed a clock and a MAC address (and several Pis cloned from one SD card image can share both), and v3 and v5 are *hashes of a name*, so equal input gives an equal id by design. (v7 is time-ordered and has less randomness than v4. It is not recommended here.)
- **Do not** generate ids with a home-made or seeded PRNG (for example one seeded from the clock at boot). Several Pis booting at similar times would produce the same sequence. `Uuid::new_v4()` uses the OS random source, which is correct.
- **Do not** cache or pre-generate ids in the SD card image. Every Pi must create its ids at runtime, so Pis cloned from one image do not share a sequence.
- Generate the id **once, when the event is created**, and store it with the event. Never generate it again for a retry.

The server accepts any well-formed UUID and does not check the version, so a non-v4 id is **not** rejected. Nothing on the server will warn you if a Pi uses the wrong kind. If one did produce a duplicate id, the result is safe but visible: a different event with the same id gets a `409` (see the table) and the Pi keeps the event.

The most important rule: send the same bytes every time

The server hashes **the raw bytes of the body**, not the parsed JSON. So:

- **Serialise the event to JSON once, keep those exact bytes, and send those same bytes on every retry.** Do not rebuild the JSON for a retry. Different key order, spacing or number formatting produces different bytes, a different hash, and a `409` (see below).
- Compute the hash over **exactly the bytes you pass to the HTTP client**, after any encoding, and before the HTTP layer. (HTTP compression such as gzip is not used by this protocol. Do not compress the request body.)
- The simplest robust design: when an event happens, write `<event_id>.json` to local disk (the final request body), and have the sender read that file and POST its bytes. The file is both the retry queue and the source of the hash.

Response

Success: `200`

The body is JSON (`Content-Type: application/json`):

```
{
  "event_id": "49ab6e94-d7ae-4952-a0d2-d0fe89d25771",
  "content_hash": "d8782efe5370801eb5d10149482fb6cdca6ad8f6826baefb4b4a295cf5d1239a",
  "duplicate": false
}
```

Field	Meaning
`event_id`	The id the server stored (lowercase, hyphenated).
`content_hash`	Lowercase hex SHA-256 of the request body bytes the server received. 64 characters.
`duplicate`	`false`: this request stored the event. `true`: the event was **already** stored by an earlier request (for example you are retrying because the first reply was lost).

`200` is returned for both a first delivery and a repeat. The Pi does not need to treat them differently: in both cases the event is stored. (`201` is deliberately not used, so that the Pi has only one success code to check.)

The Pi must verify the receipt before deleting its copy of the event:

```
delivered  ==  status == 200
           and response.event_id     == the event_id it sent
           and response.content_hash == sha256_hex(the bytes it sent)
```

If the status is `200` but the hash does **not** match, the data was changed in transit or something is wrong. Do **not** delete the event. Log it loudly and retry; if it persists, it is a bug to investigate.

Errors and what the Pi should do

Error bodies are short **plain text**, not JSON.

Status	Meaning	Retry?
`200`	Stored (new or duplicate). Verify the hash as above.	No, once verified
`400`	Malformed JSON, missing field, `event_id` not a UUID, or bad `date`.	**No.** The same bytes will always fail. Fix the bug. Keep the event aside (dead-letter) and alert.
`401`	Missing/wrong `X-Collector-Token` (from nginx).	**No** until the token is fixed. Keep the event.
`403`	Wrong HTTP method (from nginx).	No. A Pi bug.
`404`	Wrong URL.	No. A Pi bug.
`409`	`event_id` already exists with **different** content, or the stored event predates this protocol and cannot be compared.	**No.** It means the Pi reused an id for different bytes (see “Pitfalls”). Keep the event aside

Status	Meaning	Retry?
		and alert. Never answer a `409` by silently dropping the event.
`413`	Body over 20 MB.	No. Send smaller images.
`422`	`camid` is not a registered camera.	**Later**: it succeeds once the camera is registered on the server, with the **same bytes and event_id**. Keep the event and retry slowly.
`429`	Too many requests (nginx rate limit, 10 per second per IP).	Yes, after a delay.
`500`, `502`, `503`, `504`	Server-side or proxy failure (database, disk, restart).	**Yes**, with backoff. The same bytes are safe to resend.
no response / timeout / connection error	The request may or may not have been stored.	**Yes**, with backoff, with the **same bytes**. This is exactly the case the protocol is designed for.

Why retrying after a timeout is safe

If a request times out, the server may have stored the event and only the reply was lost. Resending the same bytes returns `200` with `"duplicate": true` and the same `content_hash`, and **no second copy is stored**, so the Pi cannot create duplicates by retrying. The check is safe even if two retries arrive at the same instant.

Retry behaviour (recommended)

- Keep every undelivered event on disk, and keep a queue ordered oldest first. The queue must survive a reboot or power loss.
- Use **exponential backoff with jitter**, e.g. 2 s, 4 s, 8 s … capped at a few minutes, so that a recovering server is not flooded by every Pi at once.
- Use a request timeout (e.g. 30-60 s). Large bodies over a slow link need time, but nginx closes the connection if the body upload stalls for 60 s.
- Delete the local copy **only** after a verified `200` (status, `event_id` and hash all match).
- Events do not need to be delivered in order. The server orders by `date`.
- Do not send many events in parallel on a weak link. One at a time is fine.

Computing the hash

SHA-256 over the raw body bytes, written as **lowercase hexadecimal**.

Test vectors, so you can check your implementation:

Input bytes	SHA-256 (lowercase hex)
`abc` (3 bytes)	`ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`
empty	`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`

On the command line: `sha256sum event.json`.

Example (Rust)

This compiles and runs against the collector as written. It needs these crates (versions are the ones it was tested with; newer compatible versions should work):

```
[dependencies]
uuid = { version = "1", features = ["v4"] }
sha2 = "0.10"
hex = "0.4"
base64 = "0.22"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
reqwest = { version = "0.12", default-features = false, features = ["blocking", "json", "rustls-tls"] }
```

```
use std::fs;
use std::path::{Path, PathBuf};
use std::time::Duration;

use base64::{engine::general_purpose::STANDARD, Engine as _};
use serde::{Deserialize, Serialize};
use sha2::{Digest, Sha256};
use uuid::Uuid;

const SERVER: &str = "https://collector.example.com/api/alpr/event";
const TOKEN: &str = "the-shared-secret";

/// The JSON the server expects. Field order does not matter, as long as the SAME
/// BYTES are resent on every retry.
#[derive(Serialize, Deserialize)]
```

```rust
struct Event {
    event_id: String,
    camid: String,
    date: String,        // RFC 3339 with an offset
    plate: String,
    image: String,       // base64 JPEG
    plate_image: String, // base64 JPEG
}

/// The server's `200` reply.
#[derive(Deserialize)]
struct Ack {
    event_id: String,
    content_hash: String,
    #[allow(dead_code)]
    duplicate: bool,
}

/// What to do with a queued event after one delivery attempt.
#[derive(Debug, PartialEq)]
enum Outcome {
    /// Stored and verified. Delete the local file.
    Delivered,
    /// Not confirmed. Keep the file and try again later (with backoff).
    Retry,
    /// Retrying the same bytes will never work (400, 401, 409, ...).
    /// Move the file aside (dead-letter) and raise an alert. Do not delete it.
    Reject,
}

/// Builds the request body ONCE and saves it to the queue directory.
///
/// `event_id` is a random (version 4) UUID. `Uuid::new_v4()` draws from the
/// operating system's random number generator, and this is the only place an id
/// is created, so no other UUID version can slip in. Never derive the id from the
/// plate, the time, or a counter.
///
/// The file is written under a temporary name and then renamed, so a power cut
/// cannot leave a half-written file that would later hash differently.
fn build_event(
    queue_dir: &Path,
    camid: &str,
    date_rfc3339: &str,
    plate: &str,
    scene_jpeg: &[u8],
    plate_jpeg: &[u8],
) -> std::io::Result<PathBuf> {
    let event = Event {
        event_id: Uuid::new_v4().to_string(), // lowercase, hyphenated
        camid: camid.to_string(),
        date: date_rfc3339.to_string(),
        plate: plate.to_string(),
        image: STANDARD.encode(scene_jpeg),
        plate_image: STANDARD.encode(plate_jpeg),
    };
    let body = serde_json::to_vec(&event).expect("an Event always serialises");

    let final_path = queue_dir.join(format!("{}.json", event.event_id));
    let tmp_path = queue_dir.join(format!("{}.tmp", event.event_id));
    fs::write(&tmp_path, &body)?;
    fs::rename(&tmp_path, &final_path)?; // atomic on the same filesystem
    Ok(final_path)
}

/// Lowercase hex SHA-256, the same as `sha256sum`.
fn sha256_hex(bytes: &[u8]) -> String {
    hex::encode(Sha256::digest(bytes))
}

/// Sends one queued event and says what to do with it.
fn deliver(client: &reqwest::blocking::Client, queue_file: &Path) -> Outcome {
    // The bytes on disk ARE the request body. Never rebuild them for a retry.
    let Ok(body) = fs::read(queue_file) else {
        return Outcome::Retry;
    };

    // The id comes from the saved bytes, so this also works after a reboot.
    let Ok(sent) = serde_json::from_slice::<Event>(&body) else {
        return Outcome::Reject; // a corrupt queue file will never succeed
    };
```

```
    let response = match client
        .post(SERVER)
        .header("Content-Type", "application/json")
        .header("X-Collector-Token", TOKEN)
        .body(body.clone()) // the stored bytes, unchanged
        .send()
    {
        Ok(response) => response,
        // Timeout or no connection: the event may or may not have been stored.
        // Resending the same bytes is exactly what the protocol is for.
        Err(_) => return Outcome::Retry,
    };

    match response.status().as_u16() {
        200 => match response.json::<Ack>() {
            Ok(ack) if ack.event_id == sent.event_id && ack.content_hash == sha256_hex(&body) => {
                Outcome::Delivered
            }
            // 200 but the receipt does not match, or it is unreadable.
            // Never delete the event in this case.
            _ => Outcome::Retry,
        },
        422 | 429 => Outcome::Retry,
        s if s >= 500 => Outcome::Retry,
        _ => Outcome::Reject, // 400, 401, 403, 404, 409, 413, ...
    }
}

fn main() {
    let queue = PathBuf::from("queue");
    fs::create_dir_all(&queue).unwrap();
    let client = reqwest::blocking::Client::builder()
        .timeout(Duration::from_secs(60))
        .build()
        .unwrap();

    // When a vehicle is detected: build and save the event once.
    let file = build_event(&queue, "CAM1", "2026-10-30T10:00:00+08:00", "ABC1234",
                           &[0xff, 0xd8, 0xff, 0xd9], &[0xff, 0xd8, 0xff, 0xd9]).unwrap();

    // The sender loop: try, and act on the outcome.
    match deliver(&client, &file) {
        Outcome::Delivered => fs::remove_file(&file).unwrap(),
        Outcome::Retry => { /* leave it in the queue; try again after a backoff */ }
        Outcome::Reject => { /* move it to a dead-letter directory and alert */ }
    }
}
```

Things this example does on purpose:

- **The event is built once and saved to a file** (`build_event`); `deliver` then reads those bytes back and sends them unchanged. A retry, even after a reboot, therefore sends the same bytes and gets the same hash. The `event_id` is read back from the saved bytes, not kept in memory.
- **The file is written to a `.tmp` name and renamed**, so a power cut cannot leave a half-written queue file.
- **`Outcome` is an enum** so the compiler makes the sender loop handle all three cases. The file is deleted only on `Delivered`.
- **`Retry` and `Reject` never delete the file.** `Reject` means "retrying the same bytes cannot work": move the file to a dead-letter directory and alert someone.
- **`event_id` comes only from `Uuid::new_v4()`**, so it is always a version 4 UUID.

Pitfalls

- **Rebuilding the JSON on retry.** Serialising the same struct again may not give the same bytes (for example if a field changes, or a map's key order differs between runs), and regenerating the event may change something else (a fresh timestamp, a new `event_id`). A new `event_id` means a **second event**. The same `event_id` with different bytes means `409`. Always resend the saved bytes.
- **Reusing an `event_id` for a different event.** Generate a new random (v4) UUID for every new event, and see "The `event_id`" above for what not to do. Never derive it from something that can repeat (like the plate text or a counter that resets after a reboot).
- **Different spellings of one UUID.** `ABC...` and `abc...` are the same UUID to the server, but they are different bytes. Always send the exact bytes you hashed, and keep the same spelling on retries.
- **Hashing the wrong thing.** Hash the bytes that go on the wire, not the struct, not the pre-base64 images, and not the response.
- **Compressing the request body.** The server does not decompress it.
- **Treating `409` as "already delivered".** It is not. `duplicate: true` on a `200` is how the server says "already delivered". A `409` means your data and the stored data disagree.

Quick manual test

```
BODY='{"event_id":"11111111-2222-4333-8444-555555555555","camid":"CAM1","date":"2026-10-30T10:00:00Z","plate":"ABC1234"
        9j/4AAQ","plate_image":"/9j/4AAQ"}'
echo -n "$BODY" | sha256sum        # what the Pi computes
for i in 1 2; do                   # the second reply says "duplicate": true
  curl -s -X POST https://collector.example.com/api/alpr/event \
       -H "X-Collector-Token: $TOKEN" -H "Content-Type: application/json" \
       --data-binary "$BODY"; echo
done
```

(Use `--data-binary`, not `-d`: `-d` can strip newlines, which would change the bytes and therefore the hash. Note `echo -n` so that no trailing newline is added to the hashed text.)

Existing data

Events stored before this protocol existed (for example those copied by `migrate_data`) have no hash. They can never be reported as duplicates. If a Pi sends the `event_id` of one of those rows, it gets `409`. In practice this does not happen, because those rows have ids that were not issued by a Pi.